

Κεφάλαιο 15 (Συναρτησιακός Προγραμματισμός, Haskell και συναρτησιακές επιρροές στην Python)

Ερώτημα 1

Δίνεται η ακόλουθη συνάρτηση σε Haskell.

```
foo :: [Int] -> Int
foo [] = 1
foo (x:xs) = if x `mod` 2 == 0 then foo xs else x * foo xs
```

1. Τι σημαίνει η πρώτη γραμμή της συνάρτησης;
2. Τι θα επιστρέψει η κλήση της ως εξής: `foo [1,2,3,4,5,6,7]`

Απάντηση:

1.
Η γραμμή:

`foo :: [Int] -> Int`

σημαίνει ότι η συνάρτηση δέχεται ως όρισμα μια λίστα ακεραίων και επιστρέφει έναν ακέραιο.

2.
Η κλήση της συνάρτησης:

`foo [1,2,3,4,5,6,7]`

θα επιστρέψει το γινόμενο των περιττών όρων της λίστας, δηλαδή $1 \cdot 3 \cdot 5 \cdot 7$ που είναι ίσο με 105

Ερώτημα 2

Δίνεται ο ακόλουθος κώδικας στη Haskell:

```
foo :: Integer -> Integer
foo 0 = 1
foo x = x * foo (x - 1)

bar :: [Integer] -> Integer
bar [] = 1
bar (h : t) = h * bar t
```

1. Τι κάνει η συνάρτηση `foo` και τι η συνάρτηση `bar`;
2. Ποιο θα είναι το αποτέλεσμα για κάθε μια από τις ακόλουθες κλήσεις:
 - a) `foo 4`
 - b) `bar [1,2,3,4,5]`
 - c) `foo (bar [1,2,3])`
 - d) `foo bar [1,2,3]`

Απάντηση:

1.
Η συνάρτηση `foo` επιστρέφει το παραγοντικό ενός ακεραίου αριθμού `x`, δηλαδή το γινόμενο όλων των ακεραίων από το 1 μέχρι και το `x`. Η συνάρτηση `bar` επιστρέφει το γινόμενο των στοιχείων μιας λίστας.

2.
`ghci> foo 4`
`24`
`ghci> bar [1,2,3,4,5]`

```
120
ghci> foo (bar [1,2,3])
720
ghci> foo bar [1,2,3]
```

Επιστρέφει σφάλμα (error) διότι η συνάρτηση foo δέχεται 1 μόνο όρισμα ενώ της περνάμε 2 ορίσματα

Ερώτημα 3

Δίνεται ο ακόλουθος κώδικας σε Haskell:

```
foo :: (Ord a) => [a] -> [a]
foo [] = []
foo (x:xs) =
  let left = foo [a | a <- xs, a <= x]
      right = foo [a | a <- xs, a > x]
  in left ++ [x] ++ right
```

1. Ποια είναι η λειτουργικότητα του κώδικα;

2. Αν ο κώδικας έχει τοποθετηθεί σε ένα αρχείο με όνομα `haskell_erotima3.hs`, πως μπορεί να φορτωθεί και να κληθεί μέσα από το `ghci`; Δώστε ένα παράδειγμα κλήσης της συνάρτησης `foo` και των αποτελεσμάτων που αναμένεται να ληφθούν.

Απάντηση:

```
1.
Η συνάρτηση foo δέχεται μια λίστα με στοιχεία που μπορούν να συγκρίνονται μεταξύ τους
(μικρότερο, ίσο, κ.λπ.) και τα ταξινομεί σε αύξουσα σειρά σύμφωνα με τον αλγόριθμο της γρήγορης
ταξινόμησης (quicksort).

2.
$ ghci
ghci> :load haskell_erotima3.hs
ghci> foo [4,5,2,3,1]
[1,2,3,4,5]
```

Ερώτημα 4

1. Υλοποιήστε στη Haskell τη συνάρτηση

`sumUpTo n`

που να υπολογίζει με αναδρομή το άθροισμα $1 + 2 + \dots + n$.

2. Υλοποιήστε στη Haskell τη συνάρτηση

`power n k`

που να υπολογίζει με αναδρομή την ύψωση σε δύναμη n^k

Απάντηση:

```
sumUpTo :: Integer -> Integer
sumUpTo 1 = 1
sumUpTo n = n + sumUpTo (n - 1)

power :: Integer -> Integer -> Integer
power n 0 = 1
power n k = n * power n (k - 1)
```

Ερώτημα 5

Ποια θα είναι τα αποτελέσματα των ακόλουθων εντολών στο ghci;

1. ghci> (\x -> x * 2) 3
2. ghci> (\x y -> 2 * x + y) 1 2
3. ghci> map length ["arta", "ioannina", "preveza"]
4. ghci> map (*2) [1,2,3]
5. ghci> map (\x -> x+1) [1,2,3]
6. ghci> [1,3..10]
7. ghci> take 5 [0,5..]
8. ghci> filter even [1..10]
9. ghci> 9 `div` 4 + mod 10 4
10. ghci> (/) 5 2 + (*) 2 3

Απάντηση:

```
ghci> (\x -> x * 2) 3
6

ghci> (\x y -> 2 * x + y) 1 2
4

ghci> map length ["arta", "ioannina", "preveza"]
[4,8,7]

ghci> map (*2) [1,2,3]
[2,4,6]

ghci> map (\x -> x+1) [1,2,3]
[2,3,4]

ghci> [1,3..10]
[1,3,5,7,9]

ghci> take 5 [0,5..]
[0,5,10,15,20]

ghci> filter even [1..10]
[2,4,6,8,10]

ghci> 9 `div` 4 + mod 10 4
4

ghci> (/) 5 2 + (*) 2 3
8.5
```

Ερώτημα 6

Τι θα επιστρέψουν τα ακόλουθα comprehensions της Haskell;

1. ghci> [2*x | x <- [1..5]]
2. ghci> [x+y | x <- [0,5,10], y <- [1,2]]
3. ghci> [a^2 | a <- [1..10], even a]
4. ghci> [reverse i | i <- ["arta", "ioannina", "preveza"]]
5. ghci> [i !! 1 | i <- ["arta", "ioannina", "preveza"]]

Απάντηση:

```

1.
ghci> [2*x | x <- [1..5]]
[2,4,6,8,10]

2.
ghci> [x+y | x <- [0,5,10], y <- [1,2]]
[1,2,6,7,11,12]

3.
ghci> [a^2 | a <- [1..10], even a]
[4,16,36,64,100]

4.
ghci> [reverse i | i <- ["arta", "ioannina", "preveza"]]
["atra", "aninnaoi", "azeverp"]

5.
ghci> [i !! 1 | i <- ["arta", "ioannina", "preveza"]]
"ror"

```

Ερώτημα 7

Γράψτε στη Haskell τη συνάρτηση `doubleFactorial` η που υπολογίζει το διπλό παραγοντικό ενός ακεραίου αριθμού n . Το διπλό παραγοντικό ορίζεται ως το γινόμενο όλων των ακεραίων από το 1 μέχρι και τον αριθμό n που είναι είτε άρτιοι είτε περιττοί, ανάλογα με το εάν το n είναι άρτιο ή περιττό αντίστοιχα (π.χ. το διπλό παραγοντικό του 7 είναι $1 * 3 * 5 * 7 = 105$, ενώ το διπλό παραγοντικό του 6 είναι $2 * 4 * 6 = 48$).

Απάντηση:

```

doubleFactorial :: Integer -> Integer
doubleFactorial 0 = 1
doubleFactorial 1 = 1
doubleFactorial n = n * doubleFactorial (n - 2)

```

Ερώτημα 8

Γράψτε στη Haskell μια συνάρτηση με όνομα `convert` που να δέχεται δύο ορίσματα και θα επιστρέφει το γινόμενο των δύο ορισμάτων. Το πρώτο όρισμα θα είναι ένας συντελεστής μετατροπής και το δεύτερο ένας αριθμός (για παράδειγμα αν το πρώτο όρισμα έχει την τιμή 0.62 τότε θα μπορεί να χρησιμοποιηθεί για την μετατροπή χιλιομέτρων σε μίλια καθώς 1 χιλιόμετρο είναι 0.62 μίλια). Ορίστε τη συνάρτηση `convertKm2Miles` (μετατροπή χιλιομέτρων σε μίλια) με βάση τη συνάρτηση `convert` και τη συνάρτηση `convertMiles2Km` (μετατροπή μιλίων σε χιλιόμετρα) επίσης με βάση τη συνάρτηση `convert`.

Απάντηση:

```

convert :: (Num a) => a -> a -> a
convert x y = x * y

convertKm2Miles :: Double -> Double
convertKm2Miles km = convert km 0.62

convertMiles2Km :: Double -> Double
convertMiles2Km miles = convert miles (1 / 0.62)

```

Ερώτημα 9

Κατασκευάστε στη Haskell μια συνάρτηση με όνομα `inRange` που να δέχεται τρία ορίσματα `min`, `max` και `x` (ακέραιες τιμές) και να επιστρέφει `True` ή `False` ανάλογα με το αν το `x` βρίσκεται στο διάστημα `[min, max]` ή όχι.

Απάντηση:

```
inRange :: Ord a => a -> a -> a -> Bool
inRange min max x = min <= x && x <= max
```

Ερώτημα 10

Γράψτε μια ανώνυμη συνάρτηση στη Haskell που υψώνει στον κύβο τον αριθμό που δέχεται ως όρισμα. Εφαρμόστε τη συνάρτηση σε μια λίστα αριθμών που ξεκινά από το 0 και με βήμα 0.5 φτάνει στο 5.

Απάντηση

```
ghci> f = \x -> x^3
ghci> map f [0, 0.5..5]
[0.0,0.125,1.0,3.375,8.0,15.625,27.0,42.875,64.0,91.125,125.0]
```

Ερώτημα 11

Έστω ότι δίνεται στη Haskell οι συναρτήσεις:

```
plusOne :: Int -> Int
plusOne x = x + 1

minusOne :: Int -> Int
minusOne x = x - 1
```

Χωρίς να χρησιμοποιήσετε τους τελεστές `+` και `-`, ορίστε τη συνάρτηση `addition` έτσι ώστε η:
`addition x y`
να προσθέτει τα `x` και `y`.

Απάντηση:

```
plusOne :: Int -> Int
plusOne x = x + 1

minusOne :: Int -> Int
minusOne x = x - 1

addition :: Int -> Int -> Int
addition x 0 = x
addition x y = addition (plusOne x) (minusOne y)

main :: IO ()
main = print (addition 5 7)
```

Ερώτημα 12

Α) Γράψτε μια συνάρτηση στη Haskell που να δέχεται το δείκτη μάζας σώματος (`bmi`) ενός ατόμου και χρησιμοποιώντας `guards` (ή άλλο τρόπο):

- αν `bmi <= 18.5` να επιστρέφει `underweight`
- αν `bmi <= 25` να επιστρέφει `normal`
- αν `bmi <= 30` να επιστρέφει `overweight`
- αλλιώς να επιστρέφει `obese`

Β) Γράψτε εντολή που να επιστρέφει μια λίστα με ζεύγη ονομάτων και χαρακτηρισμού (underweight, κ.λπ.) για τις ακόλουθες λίστες

```
bmis = [15.7, 28,5, 41.7]
```

```
names = ["Νίκος", "Πέτρος", "Μαρία"]
```

Απάντηση:

```
bmiCategory :: Float -> String
bmiCategory bmi
  | bmi <= 18.5 = "underweight"
  | bmi <= 25.0 = "normal"
  | bmi <= 30.0 = "overweight"
  | otherwise  = "obese"

bmis = [15.7, 28,5, 41.7]
names = ["Νίκος", "Πέτρος", "Μαρία"]
results = zip names (map bmiCategory bmis)
```

Ερώτημα 13

Δίνεται σε Python συνάρτηση που υλοποιεί τον αλγόριθμο του Ευκλείδη για την εύρεση του Μέγιστου Κοινού Διαιρέτη καθώς και η κλήση της συνάρτησης για τις τιμές 12 και 16.

```
def my_gcd(a, b):
    if b == 0:
        return a
    else:
        return my_gcd(b, a % b)

c = my_gcd(12, 16)
print(c)
```

Γράψτε ισοδύναμο κώδικα σε Haskell.

Απάντηση:

```
import Main (main)
myGcd :: Int -> Int -> Int
myGcd a b =
    if b == 0
    then a
    else myGcd b (a `mod` b)

main :: IO ()
main = print (myGcd 12 16)
```

Ερώτημα 14

Γράψτε μια συνάρτηση στη Haskell με όνομα productOdd που να δέχεται μια λίστα ακεραίων (Int) και να επιστρέφει το γινόμενο των περιττών τιμών της λίστας. Δώστε μια λύση με αναδρομή και μια δεύτερη λύση με χρήση comprehension και της συνάρτησης product.

Απάντηση:

```
productOdd :: [Int] -> Int
productOdd [] = 1
```

```
productOdd (x:xs)
  | x `mod` 2 == 1 = x * productOdd xs
  | otherwise = productOdd xs

productOdd' :: [Int] -> Int
productOdd' xs = product [x | x <- xs, odd x]
```

Ερώτημα 15

Γράψτε σε Haskell τη συνάρτηση `squareRoot` που να δέχεται μια λίστα πραγματικών τιμών και να επιστρέφει μια λίστα με τις τετραγωνικές ρίζες των μη αρνητικών τιμών της λίστας. Δώστε μια λύση με `comprehension` και μια λύση όπου θα χρησιμοποιούνται οι ενσωματωμένες συναρτήσεις `sqrt`, `map` και `filter`.

Απάντηση:

```
squareRoot :: [Double] -> [Double]
squareRoot xs = [sqrt x | x <- xs, x >= 0]

squareRoot' :: [Double] -> [Double]
squareRoot' xs = map sqrt (filter (>= 0) xs)
```

Ερώτημα 16

Γράψτε μια αναδρομική συνάρτηση στη Haskell, με όνομα `myPower`, που να υπολογίζει το x υψωμένο στη δύναμη n , υποθέτοντας ότι το n είναι μη αρνητικός ακέραιος.

Απάντηση

```
myPower :: Double -> Int -> Double
myPower x 0 = 1
myPower x n = x * myPower x (n - 1)
```

Ερώτημα 17

Γράψτε μια αναδρομική συνάρτηση στη Haskell, με όνομα `myReverse`, που να αντιστρέφει μια λίστα. Εφαρμόστε τη συνάρτηση στα στοιχεία μιας λίστας που περιέχει λίστες. Γράψτε το αναμενόμενο αποτέλεσμα.

Απάντηση:

```
myReverse :: [a] -> [a]
myReverse [] = []
myReverse (x:xs) = myReverse xs ++ [x]

main::IO()
main = print (myReverse [[1,2,3], [4,5], [6,7,8,9]])
```

Ερώτημα 18

Για κάθε μια από τις ακόλουθες περιπτώσεις μετατρέψτε τον κώδικα Haskell σε Python

```
doubleAll :: [Int] -> [Int]
doubleAll xs = map (*2) xs

onlyEven :: [Int] -> [Int]
onlyEven xs = filter even xs

applyTwice :: (a -> a) -> a -> a
applyTwice f x = f (f x)

process :: [Int] -> [Int]
```

```
process xs = map (^2) (filter odd xs)
```

Απάντηση

```
def double_all(xs):  
    return list(map(lambda x: x * 2, xs))  
  
def only_even(xs):  
    return list(filter(lambda x: x % 2 == 0, xs))  
  
def apply_twice(f, x):  
    return f(f(x))  
  
def process(xs):  
    return list(map(lambda x: x ** 2, filter(lambda x: x % 2 == 1, xs)))
```

Ερώτημα 19

Μετατρέψτε τον ακόλουθο κώδικα Haskell σε Python χρησιμοποιώντας την partial από τη βιβλιοθήκη functools.

```
power :: Int -> Int -> Int  
power x y = x ^ y  
  
square :: Int -> Int  
square = power 2  
  
square 5
```

Απάντηση:

```
from functools import partial  
  
def power(x, y):  
    return x ** y  
  
square = partial(power, 2)  
  
print(square(5))
```

Ερώτημα 20

Έστω ότι στο REPL της Python έχει δοθεί η εντολή:

```
from itertools import count, combinations, product, accumulate, cycle, islice
```

Τι θα εμφανίσει κάθε μια από τις ακόλουθες εντολές;

1. `list(islice(count(3), 5))`
2. `list(combinations([1, 2, 3], 2))`
3. `list(product(['A', 'B'], [1, 2]))`
4. `list(accumulate([1, 2, 3, 4]))`
5. `list(islice(cycle(['a', 'b']), 6))`
6. `list(islice(count(10, 2), 4))`
7. `list(accumulate([2, 3, 4], lambda x, y: x * y))`

Απάντηση:

```
>>> list(islice(count(3), 5))  
[3, 4, 5, 6, 7]  
  
>>> list(combinations([1, 2, 3], 2))
```



```
[(1, 2), (1, 3), (2, 3)]
```

```
>>> list(product(['A', 'B'], [1, 2]))  
[('A', 1), ('A', 2), ('B', 1), ('B', 2)]
```

```
>>> list(accumulate([1, 2, 3, 4]))  
[1, 3, 6, 10]
```

```
>>> list(islice(cycle(['a', 'b']), 6))  
['a', 'b', 'a', 'b', 'a', 'b']
```

```
>>> list(islice(count(10, 2), 4))  
[10, 12, 14, 16]
```

```
>>> list(accumulate([2, 3, 4], lambda x, y: x * y))  
[2, 6, 24]
```